

# 신경망 기초, 역전파, 학습 기법

퍼셉트론의 한계와 역전파 · 그래디언트 소실 · 폭발과 활성화함수 · 초기화 · 정규화와 학습률 스케줄링

Machine Learning 1 · 9주차

한국과학영재학교 (KSA)

Chapter 9

# 이번 주차 개요

## 이 챕터를 마치면

- XOR이 직선 하나로 안 풀리는 이유를 설명하고, 은닉층이 어떻게 푸는지 보여준다
- 역전파를 연쇄법칙으로부터 유도하고, 구현을 수치 미분으로 검증한다
- 그래디언트 소실·폭발을 진단하고, 활성화 함수와 초기화를 고른다
- 정규화와 학습률 스케줄링을 과적합·불안정 학습 상황에 맞게 적용한다
- **9.1 퍼셉트론의 한계, 순전파, 역전파** — XOR의 벽, 오차  $\delta$ 의 역전파, 2-2-1 손계산, numpy로 XOR 풀기
- **9.2 그래디언트 소실·폭발, 활성화 함수, 초기화** — 연쇄법칙의 곱, ReLU 계열, Xavier/He 초기화, “세 라인 방어선”
- **9.3 정규화 기법과 학습률 스케줄링** — dropout, BatchNorm, 가중치 감쇠, StepLR/코사인/warmup

세 절은 한 사슬: 그래디언트는 “연쇄법칙의 곱”(9.1) → 깊어지면 0으로 사라지거나 불어나는(9.2) → 학습을 안정적으로 끝내는 도구(9.3). Block B(CNN → RNN → 트랜스포머 → LLM)의 서막이다.

---

## 9.1 퍼셉트론의 한계, 순전파, 역전파

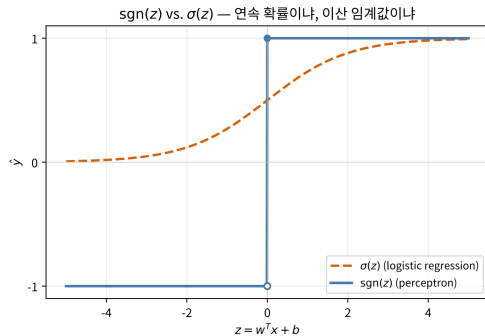
---

# 퍼셉트론: “AI의 시작”이자 “AI의 실망”

- 1958, 로젠블라트(코넬): 퍼셉트론 발표 — “컴퓨터가 배운다”, “우주 탐사 1세대 로봇”
- Chapter 2의 로지스틱회귀와 거의 동일, 시그모이드 대신 **스텝함수**

$$\hat{y} = \text{sgn}(w^T x + b)$$

- 실제로 (당시 기준) 조종사 얼굴 인식 실험까지 수행



sgn(파랑,  $z = 0$ 에서 도약) vs 시그모이드(주황, 매끄러운 전이)

**퍼셉트론 학습 규칙** (경사하강법의 선조): 틀린 예제에만 오차 방향으로 갱신,  $w \leftarrow w + yx$  — 선형분리 가능 데이터에서 **반드시 수렴**(수렴 정리)

# 1969, Perceptrons: XOR의 벽과 AI의 겨울

- **민스키·페퍼트(MIT, 1969)**: 단층 퍼셉트론은 XOR을 아무리 학습시켜도 절대 못 맞춘다 — 수학적으로 증명
- “학습률 부족”, “데이터 부족”이 아니라 **모델 구조 자체가 불가능**

**영향: 투자·논문 급감 → 1차 “AI 겨울”(1970s 후반~1980s 중반)의 직접 원인. 대중적으로는 “신경망은 쓸모없다”로 과도하게 일반화됐다.**

- 그 긴 동면기를 깨운 것: 1986년 **럼멜하트-힌튼-윌리엄스**의 **역전파** 대중화 논문
- 동면기를 깨운 것이 바로 “XOR의 벽을 넘는 단 하나의 구조” — 다음 프레임의 **은닉층**

# 퍼셉트론에서 다층 신경망(MLP)으로

- 해법: 직선 하나로 안 되면, 직선을 여러 개 겹쳐 새 공간을 만든 뒤 그 공간에서 다시 나누기
- 로지스틱회귀 = 은닉층 없는 가장 단순한 “신경망”  $a = \sigma(w^T x)$
- **MLP**: 그 사이에 **은닉층**을 하나 이상 끼워 넣음
- 은닉층 하나만 추가해도 XOR을 완벽히 풀 수 있다

| $x_1$ | $x_2$ | XOR |
|-------|-------|-----|
| 0     | 0     | 0   |
| 0     | 1     | 1   |
| 1     | 0     | 1   |
| 1     | 1     | 0   |

라벨 1인 점 (0, 1), (1, 0)과 라벨 0인 점 (0, 0), (1, 1)이 대각선으로 교차 배치 — 어떤 직선을 그어도 서로 다른 라벨이 한쪽에 섞여 들어간다.

# XOR의 부등식 모순 — 직접 세워보자

직선분리 가능하다는 말 = 어떤  $(w_1, w_2, b)$ 가 다음을 **모두** 만족:

$$w_2 + b > 0 \quad (0, 1) \quad w_1 + b > 0 \quad (1, 0) \quad b \leq 0 \quad (0, 0) \quad w_1 + w_2 + b \leq 0 \quad (1, 1)$$

- ① 첫 두 식을 더하면  $w_1 + w_2 + 2b > 0$
- ② 셋째 식( $b \leq 0$ )과 결합  $\Rightarrow w_1 + w_2 + b > -b \geq 0$
- ③ 넷째 식은  $w_1 + w_2 + b \leq 0$ 이라 주장 — **모순**

## 결론

부등식 네 개는 어떤  $(w_1, w_2, b)$ 로도 동시에 성립 불가. 퍼셉트론에 필요한 것은 **비선형 결정경계**.

## 은닉층이 실제로 하는 것: “새 공간” 만들기

은닉 유닛 2개를  $f_1 = \sigma(x_1 + x_2 - 1)$ ,  $f_2 = \sigma(-x_1 - x_2 + 1)$ 로 정하면:

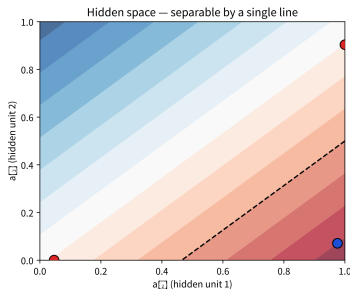
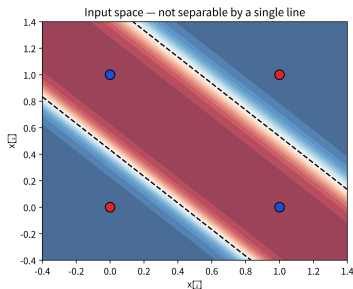
| $(x_1, x_2)$ | 라벨 | $f_1$          | $f_2$          |
|--------------|----|----------------|----------------|
| (0, 0)       | 0  | $\approx 0.27$ | $\approx 0.73$ |
| (1, 1)       | 0  | $\approx 0.73$ | $\approx 0.27$ |
| (0, 1)       | 1  | 0.5            | 0.5            |
| (1, 0)       | 1  | 0.5            | 0.5            |

- 새 공간  $(f_1, f_2)$ 에서: **라벨 1의 두 점은 한 점으로 모이고**, 라벨 0의 두 점은 좌표가 다름
- 이제 **직선 하나로 깔끔히 분리**

중요한 것은 “이런 가중치가 존재한다”는 것 — 그리고 역전파가 실제로 그 가중치를 찾는다(실습: 은닉 4개로 학습).



# “새로운 공간”이 그림으로: 결정경계



- 입력 공간 ( $x_1, x_2$ ): 네 점 사이를 비선형 곡선이 지나 — 직선 하나로 못 나눔
- 은닉 공간 ( $a_1, a_2$ ): 같은 네 점이 직선 하나로 깔끔히 분리

## 은닉층의 역할

“새 공간을 만들어, 거기서 직선으로 나누는 것.”

(실제로 학습된 망, 은닉 유닛 2개 · 20,000 에폭)

## 순전파 (Forward Propagation)

2층 신경망(입력  $\rightarrow$  은닉  $\rightarrow$  출력,  $W_1, b_1$  은닉층 /  $W_2, b_2$  출력층):

$$z_1 = W_1 x + b_1, \quad a_1 = \sigma(z_1), \quad z_2 = W_2^\top a_1 + b_2, \quad a_2 = \sigma(z_2)$$

- $z$ : **활성화 전**(pre-activation),  $a$ : **활성화 후** — 최종 예측값은  $a_2$

### 주의 — 컨벤션

$W_1 \in \mathbb{R}^{m \times d}$  (행=은닉 유닛, 열=입력)이면  $z_1 = W_1 x + b_1$ 이 자연스럽고,

PyTorch/TensorFlow의 Linear와 일치.

아래 손계산은 **전치 배치**(행=입력, 열=은닉)로  $z_1 = x^\top W_1 + b_1$ 를 쓴다 — “같은 값이 전치된 모습으로 쓰여 있는” 것이 수치 미분 검증·9.2절의 가장 흔한 혼동의 근원.

## 역전파: 연쇄법칙을 거꾸로

- 새 문제: 은닉층 가중치를 어떻게 학습할 것인가 — 은닉층에는 정답을 직접 비교할 대상이 없다
- **역전파**(rumelhart-hinton-williams, 1986): 출력층의 오차를 **연쇄법칙으로 거꾸로** 흘려보내, 각 가중치가 최종 오차에 얼마나 책임이 있는지 정확히 계산

$z_1 \rightarrow a_1 \rightarrow z_2 \rightarrow a_2 \rightarrow L$  (이 긴 사슬을 출력에서 입력 방향으로 거꾸로 따라가며 미분을 곱)

- ① **출력층 오차:**  $\delta_2 = \frac{\partial L}{\partial z_2} = (a_2 - y) \sigma'(z_2)$
- ② **은닉층 오차:**  $\delta_1 = (W_2 \delta_2) \odot \sigma'(z_1)$  ( $\odot$ : 원소별 곱)
- ③ **그래디언트:**  $\frac{\partial L}{\partial W_2} = \delta_2 a_1^\top, \quad \frac{\partial L}{\partial W_1} = \delta_1 x^\top$

## 왜 출력층 오차가 “ $a_2 - y \cdot \sigma'$ ”로 깔끔한가

**MSE**  $L = \frac{1}{2}(a_2 - y)^2$  일 때:

$$\delta_2 = (a_2 - y) \sigma'(z_2) = (a_2 - y) a_2(1 - a_2)$$

시그모이드 미분항이 **그대로** 곱된다.

**교차 엔트로피**  $L = -[y \ln a_2 + (1 - y) \ln(1 - a_2)]$  일 때:

$$\frac{\partial L}{\partial a_2} = -\frac{y}{a_2} + \frac{1 - y}{1 - a_2}$$

$\sigma'(z_2) = a_2(1 - a_2)$ 와 곱하면 **정확히 약분**.

“교차 엔트로피 + 시그모이드 출력”이 표준인 계산적 이유

$\delta_2 = a_2 - y$  — 포화 항(최대 0.25)이 붙지 않아, 예측이 0이나 1에 포화돼도 그래디언트가 상대적으로 크게 유지.

은닉층에서는 약분이 안 일어나  $\delta_1$ 에는 여전히  $\sigma'(z_1)$ 가 곱된다 — 9.2절 소실 문제의 시작점.

## “역전파”라는 이름의 의미: 오차 벡터의 재사용

- 오차  $\delta$ 는 **출력층**에서 계산되고, 은닉층 방향으로 **거꾸로 전파**되면서 각 층의 그래디언트를 만들

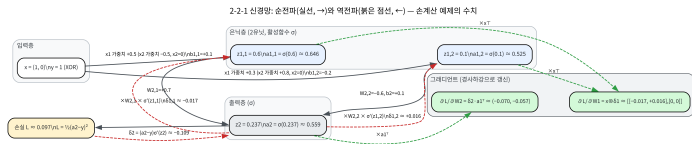
$$\delta_{1,i} = \delta_2 \cdot W_{2,i} \cdot \sigma'(z_{1,i}) \quad \Rightarrow \quad \delta_1 = (W_2 \delta_2) \odot \sigma'(z_1)$$

### 그래디언트 재사용 — 역전파는 동적 프로그래밍

각 층의  $\delta$ 를 **한 번** 계산해 두고 아래 층에 **다시 쓰는** — “같은 미분을 두 번 계산하지 않는” 알고리즘.

- 매 파라미터를  $\epsilon$ 씩 흔드는 수치 미분:  $O(W)$  전파  $\times O(W)$  파라미터 =  $O(W^2)$
- 역전파: **순전파 1회 + 역전파 1회**  $\approx O(W)$

# 순전파와 역전파의 흐름: 한 장의 그림



- 실선 = 순전파:  
 $z_1 \rightarrow a_1 \rightarrow z_2 \rightarrow a_2$  (입력 → 은닉 → 출력)
- 점선 = 역전파:  $\delta_2 \rightarrow \delta_1$  (출력 → 은닉)

역전파는 이름뿐이 아니라, 오차 값이 실제로 층을 거슬러 흘러가는 것. 각 층의 그라디언트  $\partial L / \partial W_2, \partial L / \partial W_1$ 이 여기서 만들어진다.

## 손으로 한 번: 2-2-1 순전파 + 손실

$$x = (1, 0), y = 1 \text{ (XOR(1,0)=1)}, W_1 = \begin{bmatrix} 0.5 & 0.3 \\ -0.5 & 0.8 \end{bmatrix}, b_1 = (0.1, -0.2), W_2 = (0.7, -0.6), b_2 = 0.1:$$

- **순전파:**  $z_1 = x^\top W_1 + b_1 = (0.6, 0.1), a_1 = \sigma(z_1) \approx (0.646, 0.525)$
- $z_2 \approx 0.646 \times 0.7 + 0.525 \times (-0.6) + 0.1 \approx 0.237, a_2 \approx 0.559$
- **손실:**  $L = \frac{1}{2}(a_2 - y)^2 \approx 0.097$

**역전파:**

$$\delta_2 \approx (-0.441) \times 0.559 \times 0.441 \approx -0.109$$

$$\delta_1 = W_2 \delta_2 \odot \sigma'(z_1) \approx (-0.076, 0.065) \odot (0.229, 0.249) \approx (-0.017, 0.016)$$

행=입력 컨벤션:  $\partial L / \partial W_1 = x \otimes \delta_1, x_2 = 0$ 이므로 두 번째 행은 0.

## 2-2-1 손계산: 그래디언트와 한 스텝 갱신

$$\frac{\partial L}{\partial W_2} \approx \delta_2 a_1 \approx (-0.070, -0.057), \quad \frac{\partial L}{\partial W_1} \approx \begin{bmatrix} -0.017 & 0.016 \\ 0 & 0 \end{bmatrix}$$

- 학습률  $\alpha = 0.5$ 로 **한 스텝** 갱신  $\Rightarrow$  손실  $0.097 \rightarrow 0.087$
- **핵심**: 은닉층 가중치  $W_1$ 까지 **정확히 한 번의 역전파**로 갱신됐다

### 왜 이 계산이 중요하나

PyTorch/TensorFlow는 `.backward()` 한 줄로 이 모든 미분을 자동 계산(자동 미분, 1986 이후의 표준). 하지만 내부에서 연쇄법칙이 정확히 무엇을 하는지 모르면, 9.2절의 소실 문제를 **원인 진단**할 수 없다.



# 수치 미분 검증: “역전파가 맞다”를 기계로

**수치 미분**(numerical gradient checking): 가중치를 아주 작게 ( $\epsilon = 10^{-6}$ ) 흔들어서 손실 변화량으로 미분을 직접 재봄

$$\frac{\partial L}{\partial w} \approx \frac{L(w + \epsilon) - L(w - \epsilon)}{2\epsilon}$$

| 파라미터                             | 역전파     | 수치 미분   | 차이                    |
|----------------------------------|---------|---------|-----------------------|
| $\partial L / \partial b_2$      | -0.1087 | -0.1087 | $6.6 \times 10^{-12}$ |
| $\partial L / \partial W_{2,1}$  | -0.0702 | -0.0702 | $1.5 \times 10^{-11}$ |
| $\partial L / \partial b_{1,1}$  | -0.0174 | -0.0174 | $6.3 \times 10^{-12}$ |
| $\partial L / \partial W_{1,11}$ | -0.0174 | -0.0174 | $6.3 \times 10^{-12}$ |

모든 차이  $10^{-11}$  수준 = 반중심 차분 오차 한계  $O(\epsilon^2) + O(10^{-16})$ 와 정확히 일치. 새 구조를 직접 구현할 때 실무에서 흔히 쓰는 디버깅 기법. **주의:**  $W_1$ 의 행/열 컨벤션이 코드와 다르면 그래디언트가 **전치**되어 “맞는 값인데 왜 안 맞지?” — 역전파가 틀린 게 아니라 컨벤션이 다른 것.

# 실습: numpy로 2층 신경망 + XOR

```
import math, random

def sigmoid(x):
    return 1 / (1 + math.exp(-x))

def sigmoid_prime(x):
    s = sigmoid(x)
    return s * (1 - s)

def two_layer_forward(x, W1, b1, W2, b2):
    z1 = [sum(W1[i][j] * x[j] for j in range(len(x))) + b1[i]
          for i in range(len(b1))]
    a1 = [sigmoid(v) for v in z1]
    z2 = sum(W2[i] * a1[i] for i in range(len(a1))) + b2
    return sigmoid(z2), (x, z1, a1, z2, sigmoid(z2))

def two_layer_backward(y_true, cache, W2):
    x, z1, a1, z2, a2 = cache
    delta2 = (a2 - y_true) * sigmoid_prime(z2)
    delta1 = [W2[i] * delta2 * sigmoid_prime(z1[i]) for i in range(len(z1))]
    grad_W2 = [delta2 * a1[i] for i in range(len(a1))]
    grad_W1 = [[delta1[i] * x[j] for j in range(len(x))]
                for i in range(len(delta1))]
    return grad_W1, delta1, grad_W2, delta2
```

## 같은 데이터, 다른 운명: 로지스틱회귀 vs 2층 신경망

같은 XOR 4개, 같은 경사하강법( $\alpha = 1.0$ , 5000 에폭):

| 모델              | 최종 예측                        | 손실     |
|-----------------|------------------------------|--------|
| 로지스틱회귀 (은닉층 없음) | [0.500, 0.500, 0.500, 0.500] | 0.125  |
| 2층 신경망 (은닉 4개)  | [0.016, 0.977, 0.976, 0.030] | 0.0003 |

- 로지스틱회귀: 네 점을 **전부 0.5(완전 불확신)**으로 예측한 채 0.125에서 **멈춤** — 앞에서 본 부등식 모순이 수렴을 원천 차단
- 2층 신경망: 0.0003까지 내려 네 점을 정확히 분리

### 구조의 차이가 운명을 정한다

“은닉층이 왜 필요한가”를 가장 극적으로 보여주는 최소 예시.

# “XOR을 풀었다”는 말이 성립하려면: 초기화 + 학습률

XOR 학습 성공 = (1) 무작위 초기화가 적절 + (2) 학습률이 적절.

**0-초기화** (대칭성 문제): 모든 은닉 유닛이 같은 입력  
· 같은 업데이트를 받아 **영원히 같은 값** 유지

0-초기화: 예측 [0.5, 0.5, 0.5, 0.5], 손실 0.125  
(멈춤)

출력층이 “같은 값 4개를 합산” = 사실상 1차원. XOR을  
분리할 표현력이 **구조적으로 없음**. (9.2절 Xavier/He 해결)

**학습률의 “적당함”은 문제별로 다르다.** 4점 XOR에서  $\alpha = 0.5$ 가 동작하는 것은 아니다.

**학습률** (같은 구조, 무작위 초기화, 시드 고정):

$\alpha = 0.01$ : 손실 0.124 (사실상 무학습)

$\alpha = 0.5$  : 손실 0.0003 (성공)

$\alpha = 2.0$  : 손실 0.0001 (성공, 약간 빠름)

0.01은 50,000 에폭이 돼야 겨우 수렴 — “느리다”가  
아니라 “못 배운다”. 2.0은 이 예제에선 수렴하나, 데이터가  
크면 발산.

$\alpha = 0.5$ 가 동작한다고 10만 데이터의 CNN에서

## 자주 하는 실수 (9.1)

- ① **가중치를 0으로 초기화** — 모든 은닉 유닛 출력이 동일해 대칭이 안 깨지고, XOR 포함 어떤 선형비분리 문제도 불가 (실험의  $[0.5, 0.5, 0.5, 0.5]$ ). 무작위 초기화는 잡음이 아니라 **대칭을 깨는 필수 절차**.
- ② **“작게 하면 안전”** —  $\alpha$ 가 너무 작으면 사실상 수렴 불가, 너무 크면 발산(Ch. 2.1). 새로운 구조에서는  $\{0.01, 0.1, 1.0, 10\}$  후보를 먼저 비교 (9.3의 스케줄링은 이 “적정값”을 학습 진행에 따라 조절).
- ③ **행/열 컨벤션 혼동** —  $W_1$ 이 “행=입력, 열=은닉”인지에 따라 그래디언트가 **전치**. “역전파가 틀렸다”고 하기 전에, 수치 미분 코드와 역전파 코드가 **동일 컨벤션**인지 확인.
- ④ **역전파 = “새로운 수학” 오해** — 연쇄법칙의 재사용 (동적 프로그래밍)일 뿐, 새 미분 정리가 아니다.

## 자주 묻는 질문 (9.1)

- **Q. 은닉층이 하나로 충분한가? 보편근사정리**(hornik, 1991): 은닉층 **하나**(뉴런 수 충분히 많으면)로도 이론적으로 어떤 연속함수든 근사 가능. 하지만 “가능하다”  $\neq$  “학습이 쉽다” — 실전은 더 깊은 쪽이 같은 표현력을 **훨씬 적은 뉴런 수**로 달성 (CNN·트랜스포머의 “깊이” 효율성).
- **Q. 역전파가 매번 같은 미분을 계산한다는 확신은? 수치 미분**으로  $-\frac{L(w + \epsilon) - L(w - \epsilon)}{2\epsilon}$  가 역전파 값과 소수점 몇 자리 내면 일치.
- **Q. 연쇄법칙인데 왜 1986년에야?** 아이디어는 1974년 워브루브스키 박사논문까지 거슬림 — 1986 논문은 **대중화**: (a) 최우도 추정과 연결, (b) 문자 인식 실험, (c) “은닉층을 학습할 수 있다”의 메시지.
- **Q. 순전파 vs 역전파 계산량? 동급** — 둘 다  $O(\text{총가중치})$ . 한 에폭 = 순전파 1회 + 역전파 1회  $\approx 2 \times$  순전파.

## 확인 문제 (9.1)

- ① **XOR의 부등식 모순.**  $w_2 + b > 0, w_1 + b > 0, b \leq 0, w_1 + w_2 + b \leq 0$  — 첫 두 식을 더해 셋째 식과 모순을 만들어, 이 네 식을 모두 만족하는  $(w_1, w_2, b)$ 가 **존재하지 않음**을 보여라.
- ②  **$\delta_1$ 의 재사용.**  $\delta_1 = (W_2 \delta_2) \odot \sigma'(z_1)$ 에서  $\delta_2$ 가 출력층에서 이미 계산돼 **재사용** 된다는 것이 역전파가 “동적 프로그래밍”인 이유. 10층 망에서 모든  $\delta_l$ 을 처음부터 다시 계산하면 몇 번의 미분이 필요한가, 재사용하면 몇 번인가?
- ③ **교차 엔트로피의 약분.** 시그모이드 + 교차 엔트로피면  $\delta_2 = a_2 - y$ 로 미분항이 약분. MSE 때의  $\delta_2$ 와 비교하고, 그 차이가 “포화된 예측에서도 그래디언트가 살아있는” 것과 어떻게 연결되는지 한 문장으로.

---

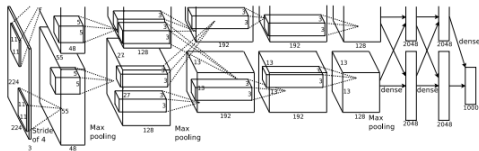
## 9.2 그래디언트 소실·폭발, 활성화함수, 초기화

---



# 시그모이드의 숨은 함정: 그래디언트 소실

- 역전파는 이론적으로 층이 몇 개든 그래디언트를 계산 가능
- 실제로 10개, 20개로 깊게 쌓으면: **입력 가까운 층은 학습이 거의 진행되지 않음** — 깊을수록 오히려 못 배운다
- 1990년대: 르쿤의 MNIST 네트워크도 **2~3층**에 불과, “딥러닝”이라는 말 자체가 2006년 힌턴까지 사실상 사라짐



AlexNet (2012, ImageNet 승리): 5 합성곱 + 3  
완전연결 = 8층 — 그 “깊음” 자체가 뉴스였다.

원인:  $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ 의 **최댓값은  $z = 0$ 에서 0.25**.  
역전파는 층을 거슬러 이 미분값을 곱해서 전달.

$$\underbrace{0.25 \times 0.25 \times \dots}_{10 \text{ 개 층}} \approx 9.5 \times 10^{-7}$$

→ **그래디언트 소실(vanishing)**: 신호가 사실상 0.  
반면 가중치 초기화가 잘못되면 층을 거칠수록 오히려  
지수적으로 커지는 **그래디언트 폭발(exploding)**도 일어남.

## 왜 “곱”인가 — 할인율 비유

- $L$ 이 층 1의  $W_1$ 에 영향을 미치는 경로:  $W_1 \rightarrow z_1 \rightarrow a_1 \rightarrow z_2 \rightarrow \cdots \rightarrow L$  — 연쇄법칙이 이 사슬을 따라 미분을 **연속으로 곱**
- 곱 연산은 **할인율**처럼: 각 층이 신호의 일부만 전달하면 10층에  $0.25^{10}$ 으로 지수 감소
- 비유: 10명이 순서대로 “들린 것의 25%만” 다음 사람에게 속삭이면 — 10번째에게는 100만분의 1도 안 남음

### 핵심

소실은 “느리다”가 아니라 **“지수적으로 0에 수렴한다”**.

소실과 폭발은 **같은 연쇄법칙 곱의 두 얼굴**: 각 층의 “승수”가 1보다 작으면 소실, 크면 폭발. 이 승수는 활성화함수( $\sigma'$ )와 가중치( $W_l$ )의 크기로 — 이번 절의 “활성함수”와 “초기화·정규화”가 바로 이 승수를 제어하는 것.

## L개 층의 곱 형태 + 손으로 체감

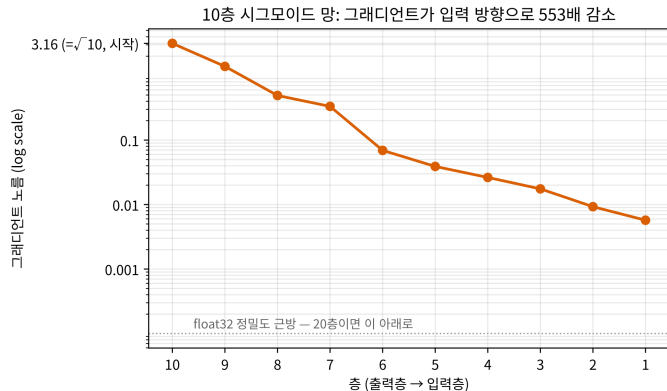
$$\frac{\partial L}{\partial W_1} \propto \prod_{l=2}^L \sigma'(z_l) \cdot W_l$$

- 각  $\sigma'(z_l) < 1$  (시그모이드)  $\Rightarrow$  지수적으로 0  $\rightarrow$  **소실** | 각 항이  $> 1$  (가중치 큼)  $\Rightarrow$  지수 증가  $\rightarrow$  **폭발**

| 상황                    | 값                              | 의미                             |
|-----------------------|--------------------------------|--------------------------------|
| $\sigma' = 0.2$ 3개 층  | $0.2^3 = 0.008$                | 그래디언트의 0.8%만 1층에 도달            |
| $\sigma' = 0.2$ 10개 층 | $\approx 1.0 \times 10^{-7}$   | 사실상 0                          |
| $\sigma' = 0.2$ 20개 층 | $\approx 1.05 \times 10^{-14}$ | float32 이내 $\rightarrow$ 반올림 0 |
| 승수 평균 2, 20개 층        | $2^{20} \approx 10^6$          | 백만 배 부풀림, 발산 (NaN 전조)          |

$\sigma'(z) = 0.2$ 는  $z \approx \pm 1.1$  — 시그모이드 곡선이 “비포화”인 전형 지점. 20층만 거치면 가중치가 발산하고 학습 중 NaN이 된다.

# 실습: 10층 시그모이드에서 실제로 사라지는 그래디언트



- 폭 10 시그모이드 층 10개,  $W \sim \mathcal{N}(0, 1)$
- 출력층 3.16 → 입력층 0.0057, **약 550배 감소**
- $0.25^{10} \approx 9.5 \times 10^{-7}$ 은 활성 미분만 곱한 극단 하한 — 가중치 곱( $\|W\| \approx \sqrt{10}$ )이 일부 보상해 550배로 관측
- 실제 평균  $\sigma'(z)$ 는 0.13~0.19 —  $z$ 가  $\pm 2 \sim 6$ 까지 벌어진 **포화** 뉴런이 많기 때문

시그모이드 미분이 “0.25 이하”인 것 자체보다, **실제 데이터 위에서 포화로 평균이 0.2를 훨씬 하회하는** 것이 더 치명적 — 입력 근처 층은 사실상 그래디언트를 못 받아 학습이 멈춘 것과 다름없음. 20층이면 float32 정밀도( $\approx 10^{-8}$ ) 이내로 **그래디언트 자체가 0으로 반올림된다**.

# 활성함수가 왜 필요한가

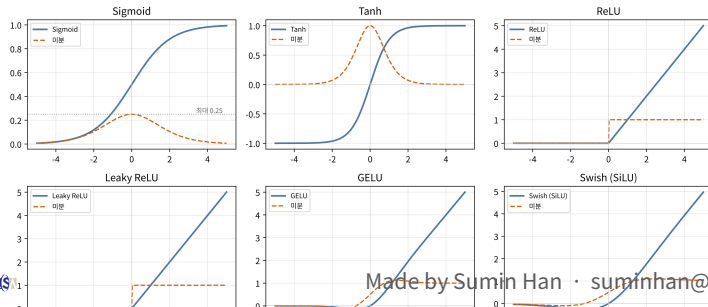
- 활성함수가 없으면:  $a_2 = W_2(W_1x) = (W_2W_1)x$  — 층을 아무리 쌓아도 한 번의 선형 변환과 같음
- 9.1에서 배운 것처럼 선형 모델로는 XOR이 **원칙적으로 불가** — 활성함수 없이는 100층을 쌓아도 영원히 못 풀음
- 활성함수는 각 층 사이에 “곡선을 접어 넣는” **비선형성**을 제공, 층이 쌓일수록 표현 가능 공간이 **지수적으로** 넓어짐

1958년 퍼셉트론의 스텝함수( $z \geq 0$ 이면 1, 아니면 0)가 역사적으로 가장 첫 번째 “활성함수” — 이 맥락에서 보면 자연스러움. 본 절의 활성함수·초기화와 9.3의 정규화는 전부 “깊은 망에서도 그래디언트가 살아서 전달되게 하려면”이라는 하나의 문제의 서로 다른 각도의 답이다.

# 기본 3종: Sigmoid / Tanh / ReLU

| 함수      | 정의                                  | 미분                   | 특징                                           |
|---------|-------------------------------------|----------------------|----------------------------------------------|
| Sigmoid | $\frac{1}{1 + e^{-z}}$              | 최댓값 0.25 ( $z = 0$ ) | 출력 (0, 1), 확률 해석 유용하나<br><b>소실에 취약</b>       |
| Tanh    | $\frac{e^z - e^{-z}}{e^z + e^{-z}}$ | 최댓값 1                | 출력 (-1, 1), sigmoid보다<br>낮지만 <b>양 끝에서 포화</b> |
| ReLU    | $\max(0, z)$                        | $z > 0: 1, z < 0: 0$ | 양수 구간 미분 <b>항상 1</b> — 층을<br>거쳐도 작아지지 않음     |

활성함수(실선)와 미분(점선) — 미분이 1 근처를 유지하는가?



여섯 가지 활성화함수(윗줄 sigmoid/tanh/ReLU, 아랫줄 Leaky ReLU/GELU/Swish)와 각 미분 — sigmoid 미분의 최댓값 0.25(점선)과 ReLU 미분의 0/1 양극화가 대비.

# Sigmoid와 Tanh의 한계

## Sigmoid — 두 가지 문제

- 1 **포화:**  $|z| \gtrsim 3$ 이면  $\sigma'(z) < 0.05$  — 깊은 망에서 소실을 가속
- 2 **중심화 문제:** 출력이 항상  $(0, 1)$ 의 양의 값  $\Rightarrow$  다음 층의 가중치가 **동일 방향**으로 움직이는 경향  $\Rightarrow$  비대칭 골짜기, 학습 느려짐

Ch.2의 로지스틱회귀에서는 확률 해석 때문에 시그모이드가 **옳지만**, 은닉층에서는 확률 해석이 불필요한데도 두 불이익만 받음.

## Tanh — 시그모이드를 $-1$ 방향으로 중심화

$$\tanh(z) = 2\sigma(2z) - 1$$

- 출력  $(-1, 1) \Rightarrow$  중심화 문제 **해소**
- 미분 최댓값 1 = 시그모이드(0.25)의 **4배**
- 1990년대 은닉층의 **사실상 표준**
- 그러나  $|z| \gtrsim 3$ 에서 여전히 포화 ( $0.985 \rightarrow$  미분  $\approx 0.03$ ) — **깊이**에는 부족

# ReLU: 2012년 이후 은닉층의 사실상 표준

- $f(z) = \max(0, z)$  — 미분이  $z > 0$ 에서 **정확히 1**이라 곱해도 줄어들지 않음
- **계산이 가장 단순**: 지수함수 없음, 비교 연산 하나
- 음수 구간 출력이 정확히 0  $\Rightarrow$  **희소(sparsity)** — 활성화된 뉴런만 신호 전달, 계산 빨라짐
- 2012년 크리스베스키 일행: “rectifier는 학습 시간을 **크게 줄인다**” — 같은 해 **AlexNet**이 ReLU를 써 ImageNet을 휩쓸며 은닉층 **사실상 표준**

자주 하는 실수: 로지스틱회귀의 습관으로 은닉층에 시그모이드를

확률 해석이 필요한 것은 **출력층**뿐. 은닉층의 기본값은 **ReLU 계열**, 출력층은 **문제 형태에 맞게**:  
회귀  $\rightarrow$  항등, 이진분류  $\rightarrow$  시그모이드, 다중분류  $\rightarrow$  softmax.



# Dying ReLU — ReLU의 아킬레스건

- $z < 0$ 일 때 미분이 **정확히 0** — 뉴런이 “죽으면” **영원히** 부활 불가 (시그모이드 포화는 미분이 작지만 0은 아님 — 성격이 다름)
- 정상 망에서는  $z \sim \mathcal{N}(0, 1)$ 의 절반이 음수라 약 50%가 “숨어” — 이 자체는 **설계값**. “dying”은 이 비율이 **비정상적으로 치솟는 것**.

**실험** (실습 노트 검증,  $y = \sin x_1 + \frac{1}{2}x_2^2$  회귀, 음수 쪽 치우친 입력, 은닉 40 ReLU, SGD, 학습률만 변경):

ReLU 학습률=0.45: 죽은 뉴런 52%  $\rightarrow$  97% (100에폭) loss 0.26 고착

Leaky ReLU 학습률=0.45: 죽은 뉴런 0% (정의상 불가) loss 0.095 계속 감소

“미분이 0”은 시그모이드 포화처럼 “느리게만” 만드는 게 아니라 **학습 경로의 한쪽을 완전히 차단**. 큰 학습률/BatchNorm 부재/outlier 입력 상황에서 죽은 뉴런이 돌이킬 수 없이 누적. (Adam 스케일 조정 + BatchNorm이  $z$  흔들림을 잡아주므로 실무에선 흔치 않지만, “죽으면 되살릴 수 없다”는 **구조적** 취약함은 변하지 않음.)

# ReLU 계열 변형 (2013년경부터)

죽는 뉴런을 “완전 사망”이 아닌 “숨을 쉰다”로 만든 것:

| 함수           | 정의                         | 한 줄 설명                                                              |
|--------------|----------------------------|---------------------------------------------------------------------|
| Leaky ReLU   | $\max(0.01z, z)$           | 음수 미분 0.01 — 죽지 않고 “숨 쉰다”                                           |
| ELU          | $\min(0, \alpha(e^z - 1))$ | 음수 구간 $-\alpha$ 로 완만 — 미분 0이 아님                                     |
| Softplus     | $\log(1 + e^z)$            | ReLU의 매끄러운 근사 — $\text{softplus}' = \sigma$                         |
| Swish (SiLU) | $z \cdot \sigma(z)$        | $z < 0$ 에서 작은 음수 허용 — ReLU보다 부드럽고, 깊은 망에서 손실 더 낮음 (2017)            |
| GELU         | $z \cdot \Phi(z)$          | “ $z$ 가 얼마나 양수적인가”의 확률로 가중 — 트랜스포머(GPT/BERT)의 표준 은닉층 활성화함수 (Ch. 12) |

GELU 기억해 두면: Ch. 12에서 “왜 GELU인가”는 바로 이번 절의 논리(미분이 1 근처 유지 + 부드러움 → 최적화 안정) 위에 얹혀 있다.